

TERESA — Braccio Unitree Z1

Diario di sviluppo, TODO list e guida di avvio

Branch attivi: [FSM_main](#) | [Z1_realsense](#) | Repo: [TERESA_ws/src/z1_vision](#)

Ultimo aggiornamento: **20 Marzo 2026**

1 Panoramica del sistema

Il sistema controlla il braccio **Unitree Z1** montato sul robot TERESA per avvicinarsi in modo autonomo al torso di un paziente. Lo stack completo è composto da:

- **Percezione:** YOLO11 pose estimation + Kalman 3D + lock tracker + body search scan
- **Pianificazione:** Workspace checker (Pinocchio + L-BFGS-B)
- **Controllo posizione:** IK Jacobiana pseudo-inversa smorzata + JointTrajectoryController
- **Controllo forza:** Impedance controller in spazio cartesiano (500 Hz, torque control)
- **Sicurezza:** keyboard safety node, stati HOMING/EMERGENCY, workspace clipping
- **Orchestrazione:** FSM a 12 stati, launch files separati per controllo e percezione

2 Branch attivi

Branch [FSM_main](#) — stack completo funzionante, base stabile

Contiene tutto il codice fino alla sessione del 2026-03-15 più il fix `ik_rot_weight` per l'orientamento EE durante JTC.

Branch [Z1_realsense](#) — feature body search scan (2026-03-20)

Estende `FSM_main` con la pipeline di scansione a griglia per trovare la miglior posizione del braccio prima di avvicinarsi al torso.

3 Architettura FSM (stato attuale — Z1_realsense)

Stati della FSM (z1_FSM.py)

Stato	Descrizione
HOMING	Stato iniziale al boot. Porta il braccio in home pose via IK/JTC, poi WAITING
WAITING	Se <code>body_scan_on_start=true</code> e scan non ancora fatto → BODY_SCANNING; altrimenti aspetta target LOCKED
BODY_SCANNING	Scansione a griglia per trovare la posizione ottimale di visione del torso
CHECKING_WORKSPACE	Verifica raggiungibilità target (thread non bloccante, Pinocchio)
APPROACHING	Invia goal IK verso il target (latchato al momento del check)
WAIT_IK_DONE	Attende <code>/ik_done=True</code> dal nodo IK
WRIST_ALIGN	Riallinea il polso perpendicolare alla superficie torso
SWITCHING_TO_TORQUE	Switch JTC → torque_controller via safe_controller_switch
IMPEDANCE_RUNNING	Impedance controller attivo: approach → hold → retract
SWITCHING_TO_JTC	Switch torque_controller → JTC
EMERGENCY_SWITCHING	Forza switch a JTC se torque attivo, poi va in HOMING
EMERGENCY	Stato bloccante, aspetta solo comando reset da tastiera
FAULT	Errore fatale (switch fallito), nodo bloccato

Topic principali (inclusi i nuovi di Z1_realsense)

Topic	Tipo	Descrizione
/torso_target_ee_locked	PoseStamped	Target EE quando tracker LOCKED
/torso_target_ee	PoseStamped	Target EE interpolato continuo
/ik_goal_pose	PoseStamped	Goal IK da FSM
/ik_enable	Bool	Abilita nodo IK
/ik_done	Bool	Conferma fine movimento JTC
/impedance_enable	Bool	Abilita impedance controller
/impedance_done	Bool	Fine ciclo impedance
/target_out_of_workspace	Bool	Target clippato dal workspace checker
/z1_fsm/state	String	Stato corrente FSM
/z1_keyboard_cmd	String	Comandi da tastiera
/torso_surface_frame	PoseStamped	Piano superficie torso (PCA)
/surface_signed_distance	Float32	Distanza TCP-superficie
/tracker_scan_mode	Bool	Attiva/disattiva scan mode nel tracker
/tracker_scan_next	Bool	Avanza al punto successivo della griglia
/torso_scan_point	Float32MultiArray	Score/dati del frame corrente [score, n_kp, conf, x, y, z]

4 DONE — Funzionalità implementate

Sessione 2026-03-15 (branch FSM_main)

✓ Pipeline base: YOLO11 + Kalman 3D + FSM IDLE/ESTIMATING/LOCKED

- ✓ IK Jacobiana pseudo-inversa smorzata + smoothstep quintico + JTC
- ✓ `realsense_surface_node`: stima piano torso da depth ROI (PCA)
- ✓ Impedance controller (500 Hz): APPROACH → HOLD → RETRACT
- ✓ `safe_controller_switch`: switch JTC ↔ torque come nodo persistente
- ✓ Workspace checker (Pinocchio + L-BFGS-B), clip target a `max_reach - 30` cm
- ✓ HOMING, EMERGENCY_SWITCHING, EMERGENCY, keyboard safety (H/ESC/-R/Q)
- ✓ Launch files: `z1_control`, `z1_perception`, `z1_realsense`
- ✓ **Fix orientamento EE**: `ik_rot_weight` portato da 0.5 a 0.8 per garantire EE perpendicolare alla superficie durante il JTC approach

Sessione 2026-03-20 (branch Z1_realsense)

Body Search Scan — nuovo file `body_search_scanner.py`

- ✓ Classe `BodySearchScanner`: FSM interna pura Python (nessun nodo ROS) con stati INIT → MOVING → RESETTING → COLLECTING → BEST_MOVING → EXITING → DONE/FAILED
- ✓ Score per punto: $detection_rate \times avg_per_frame_score$, dove $per_frame_score = (n_kp/4) \times avg_conf$
- ✓ Early stop: se score $\geq$ soglia (default 0.85) interrompe la scansione anticipatamente senza visitare i punti restanti
- ✓ Fallback al miglior punto visto se nessuno supera la soglia

Scan mode nel tracker — `z1_yolo_torso_tracker.py`

- ✓ Aggiunta FSM parallela scan mode (IDLE/COLLECTING/POINT_LOCKED), attivabile via topic `/tracker_scan_mode`
- ✓ In scan mode il tracker non entra mai in LOCKED normale: accumula frame validi e pubblica `/torso_scan_point` [score, n_kp, conf, x, y, z]
- ✓ `/tracker_scan_next`: reset del punto corrente per avanzare alla pose successiva
- ✓ Disattivando scan mode il tracker si resetta e riprende il ciclo IDLE → LOCKED normale

Stato BODY_SCANNING nella FSM — `z1_FSM.py`

- ✓ Nuovo stato `BODY_SCANNING` inserito in `WAITING` prima di aspettare il lock
- ✓ Generazione griglia $n_y \times n_z$ con orientamento look-at per ogni punto: il punto di mira è calcolato automaticamente come `home_position + lookat_distance × X_ee_home`, così in home l'orientamento è identico e spostandosi l'EE punta sempre verso lo stesso punto fisso del torso paziente
- ✓ Riutilizzo di `_orientation_for_xee()` già esistente per la rotazione minima (Rodrigues) da home verso la direzione look-at
- ✓ `_body_scan_done = False` ad ogni HOMING → ri-scansiona ad ogni nuovo ciclo

Nuovo file di configurazione config/body_search_params.yaml

- ✓ Griglia: `body_scan_center`, `ext_y/ext_z`, `ny/nz` (default: $3 \times 2 = 6$ punti, ± 20 cm Y, ± 10 cm Z)
- ✓ Look-at: `body_scan_lookat_distance`: 1.0 m
- ✓ Timing: `body_scan_point_timeout`: 4.0 s, `body_scan_min_frames`: 10
- ✓ Early stop: `body_scan_early_stop`: 0.85

5 TODO — Da fare domani (2026-03-21)**Alta priorità — test su robot reale**

- ☐ **Calibrare** `body_scan_center` in `body_search_params.yaml`: verificare che coincida con la home position reale del braccio (attualmente placeholder `[-0.09, 0.00, 0.44]`)
- ☐ **Verificare** `body_scan_lookat_distance`: misurare la distanza reale tra il braccio in home e il torso del paziente e aggiornare il parametro (default 1.0 m)
- ☐ **Testare il ciclo body scan completo**: HOMING → BODY_SCANNING (6 punti griglia) → WAITING (best pose) → CHECKING_WORKSPACE → APPROACHING → IMPEDANCE
- ☐ **Verificare l'orientamento look-at sul robot**: controllare nei log che il lookat target sia plausibile e che l'EE ruoti correttamente spostandosi in alto/basso e lateralmente
- ☐ **Verificare early stop**: eseguire con paziente ben visibile e controllare che la scan si interrompa anticipatamente ($\text{score} \geq 0.85$)
- ☐ **Testare il fix ik_rot_weight**: 0.8: verificare che l'EE sia perpendicolare alla superficie durante il JTC approach (problema del tilt precedente)

Media priorità

- ☐ **Tuning griglia**: valutare se `ext_y=0.20` e `ext_z=0.10` coprono correttamente la zona torso oppure se serve aumentare/ridurre
- ☐ **Tuning body_scan_min_frames**: verificare che 10 frame siano sufficienti per una stima affidabile dello score (aumentare se la stima è instabile, ridurre se la scan è troppo lenta)
- ☐ **Merge Z1_realsense su FSM_main** dopo validazione robot reale
- ☐ **Aggiornare launch file** per caricare anche `body_search_params.yaml` se non già incluso

Bassa priorità / futuri miglioramenti

- ☐ **Integrazione con Spot / base mobile**: usare `/target_out_of_workspace` per comandare alla base di avvicinarsi quando il target è fuori workspace
- ☐ **Logging e monitoraggio**: ROS bag recording automatico al launch con i topic principali
- ☐ **Visualizzazione RViz**: marker per griglia scan, look-at target, piano superficie, workspace reachability
- ☐ **Multi-persone**: gestione del caso in cui YOLO rileva più persone (attualmente prende la prima/più vicina)

6 Sequenza di avvio

Prerequisito: workspace ROS2 compilato e sourced.

```
cd ~/Documents/GIT_Repositories/TERESA_ws
```

```
colcon build -packages-select z1_vision && source install/setup.bash
```

Terminale 1 — Hardware robot + camera + TF

```
ros2 launch z1_vision z1_realsense.launch.py
```

Terminale 2 — Percezione (YOLO + surface)

```
ros2 launch z1_vision z1_perception.launch.py
```

Terminale 3 — Stack di controllo

```
# Avvia: safe_controller_switch + z1_ik_to_jtc + impedance (t=0s)
#         z1_FSM dopo 5s (primo stato: HOMING -> BODY_SCANNING -> ...)
ros2 launch z1_vision z1_control.launch.py
```

Terminale 4 — Keyboard safety

```
ros2 run z1_vision z1_keyboard_safety
```

Tasto	Azione
H	Invia braccio alla home position (qualunque stato)
ESC	Emergenza: stop immediato, poi HOMING
R	Reset dallo stato EMERGENCY
Q	Chiude il nodo keyboard safety

Monitoraggio

```
ros2 topic echo /z1_fsm/state # stato FSM
ros2 topic echo /torso_target_ee_locked # target YOLO
ros2 topic echo /surface_signed_distance
ros2 topic echo /torso_scan_point # score scan (durante
BODY_SCANNING)
ros2 run z1_vision fk_reader # FK end-effector
```

7 File principali

File	Ruolo
z1_FSM.py	FSM principale (12 stati, include BODY_SCANNING)
body_search_scanner.py	Classe BodySearchScanner: FSM interna griglia scan
z1_ik_to_jtc.py	IK Pinocchio → JTC action
impedance_controller_realsense.py	Controllo impedenza cartesiana 500 Hz
safe_controller_switch.py	Switch JTC ↔ torque (servizi ROS2)

File	Ruolo
<code>workspace_checker.py</code>	Raggiungibilità workspace (Pinocchio + scipy)
<code>z1_yolo_torso_tracker.py</code>	YOLO11 + Kalman 3D + lock + scan mode
<code>realsense_surface_node.py</code>	Stima piano torso (PCA depth ROI)
<code>z1_keyboard_safety.py</code>	Safety node interattivo da terminale
<code>kalman_filter.py</code>	Filtro Kalman 3D [x,y,z,vx,vy,vz]
<code>fk_reader.py</code>	Utility: FK end-effector da <code>joint_states</code>
<code>config/z1_fsm_params.yaml</code>	Parametri FSM (home pose, topic, workspace)
<code>config/z1_ik_jtc_params.yaml</code>	Parametri IK/JTC (<code>ik_rot_weight: 0.8</code>)
<code>config/z1_yolo_torso_params.yaml</code>	Parametri YOLO/Kalman/lock
<code>config/impedance_control_params.yaml</code>	Parametri impedance (guadagni, timing)
<code>config/surface_params.yaml</code>	Parametri surface node
<code>config/body_search_params.yaml</code>	Parametri griglia scan (<code>ny×nz</code> , <code>lookat</code> , <code>timeout</code> , <code>early stop</code>)
<code>launch/z1_control.launch.py</code>	Avvia stack controllo
<code>launch/z1_perception.launch.py</code>	Avvia stack percezione
<code>launch/z1_realsense.launch.py</code>	Avvia HW robot + camera + TF

8 Note tecniche

- **IK:** Jacobiana pseudo-inversa smorzata (*damped least squares*), frame LOCAL, smoothstep quintico (zero velocità a inizio/fine), joint unwrap con `_make_target_near`. `ik_rot_weight: 0.8` per EE perpendicolare alla superficie
- **Kalman:** dt adattivo dal timestamp immagine, `vel_damping = 0.9`, covarianza noise configurabile da YAML
- **Lock tracker:** varianza soglia su 20 frame + 5 check consecutivi sotto soglia
- **Body scan look-at:** `lookat = home_pos + lookat_dist × X_ee_home`. Rotazione per ogni punto griglia via `_orientation_for_xee()` (Rodrigues, rotazione minima da home). Errore angolare < 5 per variazioni di distanza ±50%
- **Impedance:** 500 Hz, PID cartesiano + joint-space polso (J4–J6), compensazione gravità+coriolis, Jacobiano LOCAL_WORLD_ALIGNED
- **URDF path** (hardcoded):
`/home/andrea/Ros2_repositories/unitree_z1_ws/install/
z1_description/share/z1_description/urdf/z1.urdf`
- **Branch:** `FSM_main` (stabile) ← base di `Z1_realsense` (feature body scan, da testare sul robot reale poi fare merge)